Digital Signal Encoder, Scrambler & Decoder Specification Report

ITT 036: Data Communications — Programming Assignment 1 | Due: 21 October 2026

Team: Roshni Dodani (roshae276@gmail.com) | Vansh Saini (vansh.sainiwork@gmail.com) | Prateek Kumar Singh (prateek...)

Repository: https://github.com/Roshae276/DATA_COMM_PROGRAMMING_ASSIGNMENT

1. System Overview & Deliverables

The project delivers an end-to-end digital transmission pipeline in **Python 3.9+** fulfilling all baseline requirements and extra-credit specifications:

- **Data Generator (`data_generator.py`):** Generates pseudo-random binary sequences or streams seeded with forced subsequences (4 or 8 consecutive zeros) for scrambling verification.
- **Optimal Palindrome Finder:** Implements **Manacher's Algorithm** to detect the longest palindromic substring in the bitstream in optimal $\mathcal{O}(N)$ time.
- **Line Encoders (`line_coding.py`, `decoder.py`):** Full implementation of **NRZ-L**, **NRZ-I**, **Manchester** (IEEE 802.3), **Differential Manchester**, and **AMI** (with forward encoders and direct decoders).
- **Scrambling Schemes (`scrambling.py`):** Features **B8ZS** ($0^8 \to 000VB0VB$) and **HDB3** ($0^4 \to 000V$ or $B00V$) with DC-balancing pulse logic and corresponding unscramblers.
- **Delta Modulation (`modulation.py`):** Analog DM encoder and staircase decoder reconstructing continuous input signals using adaptive step size Δ.
- **Extra Credit Image Decoder (`image_decoder.py`):** Reconstructs the digital stream by analyzing RGB pixel coordinates and midpoint voltage levels from exported PNG plots rather than direct raw data arrays.
- **Signal Plotter & CLI (`signal_plotter.py`, `main.py`):** Interactive menu, step waveform rendering, and dual-panel un-scrambled vs. scrambled waveform comparisons.

2. Key Assumptions & Standards

- **NRZ-L:** Bit $0 \to +1$V, Bit $1 \to -1$V (standard positive logic).
- **NRZ-I:** Transition at bit boundary for 1; no transition for 0. Default starting reference level is -1V.
- **Manchester (IEEE 802.3):** Bit $0 \to [+1, -1]$, Bit $1 \to [-1, +1]$.
- **Diff. Manchester:** Transition at start of bit for 0; no start transition for 1. Constant mid-bit transition provides clock synchronization.
- **AMI:** Bit $0 \to 0$V; Bit 1 pulses alternate polarity between $+1$V and -1V.
- **B8ZS Rule:** Pattern $0^8 \to [0, 0, 0, p, -p, 0, -p, p]$, where pulse p matches the polarity of the preceding non-zero pulse.
- **HDB3 Rule:** Pattern $0^4 \to 000V$ if odd number of 1s occurred since last replacement; $B00V$ if even (pulse B ensures DC balance).
- **Delta Modulation:** Quantization step Δ and reference are configured at runtime; bit $1 \to +\Delta$, bit $0 \to -\Delta$.
- **Image Decoder:** Plot rendered at 100 DPI with solid blue signal (`#0000FF`). Signal level is recovered by vertical center-point column scanning against baseline.

3. Algorithmic Complexity

Meets competitive coding standards; eliminates redundant loops and branch chains:

Component	Time / Space	Key Technique / Implementation
Palindrome Search	$\mathcal{O}(N)$ / $\mathcal{O}(N)$	Manacher's Algorithm with sentinels
Line Encoders/Decoders	$\mathcal{O}(N)$ / $\mathcal{O}(N)$	Single-pass linear symbol mapping
B8ZS / HDB3 Scramblers	$\mathcal{O}(N)$ / $\mathcal{O}(N)$	Lookahead pattern substitution
B8ZS / HDB3 Decoders	$\mathcal{O}(N)$ / $\mathcal{O}(N)$	Linear violation sequence scanner
Delta Modulation	$\mathcal{O}(N)$ / $\mathcal{O}(N)$	Single-pass step accumulator
Image Waveform Decoder	$\mathcal{O}(W \times H)$	2D pixel array column threshold scan

4. How to Run the Code

1. **Clone Repo:** `git clone` (see repository link in header above).
2. **Install Dependencies:** `pip install -r requirements.txt`
   Core libraries: NumPy, Matplotlib, Pillow (PIL). No MATLAB required.
3. **Run Application:** `python main.py`
4. **Interactive Workflow:** Select **1** for Digital Input (Line coding, palindrome detection, AMI scrambling, waveform plot, and extra-credit image decoding) or **2** for Analog Delta Modulation (staircase signal output).
5. **Standalone Image Decoder Test:** `python image_decoder.py`

5. References & External Sources

1. B. A. Forouzan, *Data Communications and Networking*, 5th ed., McGraw-Hill, 2013 (Ch. 4: Digital Transmission; Ch. 5: Analog Transmission).
2. W. Stallings, *Data and Computer Communications*, 10th ed., Pearson, 2013.
3. G. Manacher, "A new linear-time algorithm for finding the smallest initial palindrome of a string," *J. ACM*, vol. 22, no. 3, pp. 346–351, 1975.
4. GeeksforGeeks: Manacher's Algorithm (Linear-Time Longest Palindrome).
5. Wikipedia: Bipolar Encoding & Scrambling (B8ZS and HDB3).
6. Wikipedia: Delta Modulation Principles & Quantization Noise.
7. Python `random` Module | NumPy | Matplotlib | Pillow.

6. Declarations & Academic Integrity

AI Tools Acknowledgment: In compliance with course policy, the team declares the initial use of AI tools for scaffolding and exploration. All generated code was rigorously reviewed, debugged (notably correcting B8ZS polarity reversal edge-cases), optimized to $\mathcal{O}(N)$ using Manacher's Algorithm, and manually assembled by the team members.